



ACRE: Accelerating Random Forests for Explainability

Andrew McCrabb
mccrabb@umich.edu
University of Michigan
Ann Arbor, MI, USA

Aymen Ahmed
ahmedaj@umich.edu
University of Michigan
Ann Arbor, MI, USA

Valeria Bertacco
valeria@umich.edu
University of Michigan
Ann Arbor, MI, USA

ABSTRACT

As machine learning models become more widespread, they are being increasingly applied in applications that heavily impact people's lives (e.g., medical diagnoses, judicial system sentences, etc.). Several communities are thus calling for ML models to be not only accurate, but also explainable. To achieve this, recommendations must be augmented with explanations summarizing how each recommendation outcome is derived. Explainable Random Forest (XRF) models are popular choices in this space, as they are both very accurate and can be augmented with explainability functionality, allowing end-users to learn how and why a specific outcome was reached. However, the limitations of XRF models hamper their adoption, the foremost being the high computational demands associated with training such models to support high-accuracy classifications, while also annotating them with explainability meta-data.

In response, we present ACRE, a hardware accelerator to support XRF model training. ACRE accelerates key operations that bottleneck performance, while maintaining meta-data critical to support explainability. It leverages a novel Processing-in-Memory hardware unit, co-located with banks of a 3D-stacked High-Bandwidth Memory (HBM). The unit locally accelerates the execution of key training computations, boosting effective data-transfer bandwidth. Our evaluation shows that, when ACRE augments HBM3 memory, it yields an average system-level training performance improvement of 26.6x, compared to a baseline multicore processor solution with DDR4 memory. Further, ACRE yields a 2.5x improvement when compared to an HBM3 architecture baseline, increasing to 5x when not bottlenecked by a 16k-thread limit in the host. Finally, due to much higher performance, we observe that ACRE provides a 16.5x energy reduction overall, over a DDR baseline.

ACM Reference Format:

Andrew McCrabb, Aymen Ahmed, and Valeria Bertacco. 2023. ACRE: Accelerating Random Forests for Explainability. In *56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '23)*, October 28–November 01, 2023, Toronto, ON, Canada. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3613424.3623788>

1 INTRODUCTION

As machine learning (ML) models expand into an increasing number of domains, they are being embedded in applications entailing

evaluations that heavily impact the lives of individuals and communities. In return, consumers, policymakers, and governments are calling for more ethical, inclusive, and explainable usage of data and AI models; thus, several application domains are interested in adopting ML solutions that complement recommendations with insights into their decision process and flow. Such solutions are called eXplainable AI (XAI) models.

XAI models are capable of offering explanations summarizing how each outcome was reached. For example, an XAI model applied to loan application decisions would provide not just a recommendation like "Accept" but also the primary factors which led the model to its decision (e.g. Age > 35, credit score > 700, and income > \$50k).

There are four primary benefits of using XAI models. First, explainability makes it possible for users to ascertain that models are not only accurate, but accurate for the right reasons. Second, explanations foster greater trust between the model and its users. Third, recent legislation, like the European Union's General Data Protection Regulation [16], requires that users have a right to explanations for automated decisions that affect them. Finally, XAI models' explanations can offer users previously-unknown insights about the applications to which they are applied.

Among models which achieve these benefits, random forests (RF) are popular: in many applications their accuracy is comparable to that of neural networks [34], their structures are intuitive, and they are already widespread among many existing AI applications. Indeed, the 2021 Kaggle survey reports that ~75% of data scientists today use tree- and random forest-based models in their work [23].

Figure 1 visualizes training and inference on a random forest, using a small forest of three trees, with the goal of determining whether to accept or reject a loan application. During training, nodes are recursively constructed and connected through edges to form the trees. Each node is associated with a *condition* (e.g., "Age > 35"), based on features from the training dataset, which determine how to split the training data into subgroups, such that each subgroup contains primarily training data of the same class (e.g., Accept or Reject). During inference, test data is filtered through each tree. The RF's overall classification is simply the most common classification observed among all the forest's trees. The explainer function takes data from the training phase (made from the **All-to-One Comparison results**, or "**AtO results**"), the nodes traversed during inference, and the final classification, using them as inputs to generate an explanation associated with the classification provided.

Though highly accurate and explainable, Explainable Random Forest (XRF) models suffer from extremely slow training times, often well beyond that of modern neural networks (NNs). As a comparative example, training an NN with Pytorch or TensorFlow with a 17M-element dataset takes 17-22 hours [9], while training an RF with 1,000 trees on a similarly-sized 15M-element dataset exceeds 15 days [61]. Thus, systems must train RF models much

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MICRO '23, October 28–November 01, 2023, Toronto, ON, Canada

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0329-4/23/10...\$15.00

<https://doi.org/10.1145/3613424.3623788>

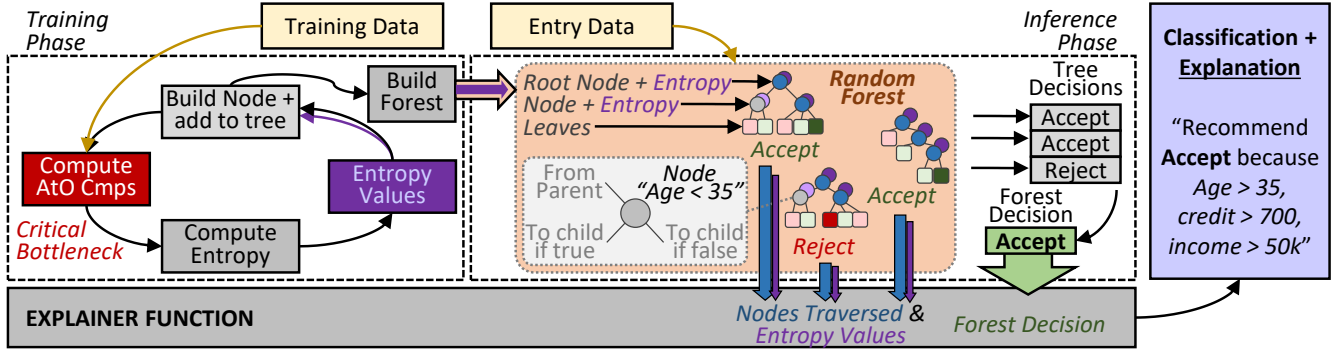


Figure 1: Overview of XRF training, inference, and the explaining function. The training phase (left) recursively builds trees from nodes, in an iterative flow leveraging AtO comparisons. The inference phase (middle) feeds entry data through the random forest’s trees, choosing the majority classification (“Accept”) as the RF’s recommendation. The explainer functions considers both AtO results from training, and paths followed and classification during inference to provide an explanation associated with the classification (right).

faster than possible today in order to be considered a viable option in large applications that have life-altering impact on their users. As an example, in judicial proceedings cases, if each case in the US had only 100 features and we used only 50 years of data, the dataset would include over two billion entries. At current training performances, it would take 5.7 years to train the corresponding RF. As another example, in the support of medical prognoses, if a dataset of only 1% of past US hospital visits were considered, over a period of a decade, and with only 100 features, the training dataset would have approximately nine billions elements. The training of that RF would take over 25 years [7][51].

Thus, in this paper, we seek to improve the performance of RF training. To this end, we first identified that the dominant operations in the RF training algorithm are AtO comparisons, which take place when the algorithm performs several entropy calculations as it seeks to identify which condition to associate to each tree’s node. In turn, the performance of AtO comparisons is limited by the communication bandwidth between compute host and memory, and by the limits of thread parallelism in carrying out these computations. Indeed, for each node, the training algorithm must compare all training data feature values many times over. Since training data often does not fit in the cache, such data must be repeatedly reloaded to complete all the comparisons against each new candidate’s splitting condition, thrashing heavily in the cache – and the associated transfer bandwidths are beyond the capabilities of even the most advanced memory systems. Additionally, as forests grow to thousands of trees, the training task can easily require 100s of billions of comparisons or more [43].

To address these challenges, we propose ACRE, a hardware accelerator for XRF applications. ACRE is a Processing-in-Memory (PIM) accelerator design leveraging new high-bandwidth memory (HBM, Generation 3 unless otherwise specified) technologies, which offer greater efficiency and higher bandwidth than traditional non-3D-stacked memory devices (e.g., DDR4). ACRE accelerator units are co-located with each memory bank in the HBM device and provide efficient support for the many comparisons entailed in random forest training. In addition, ACRE addresses a critical requirement of explainability in RF models: the explainer function must be able to

access the entropy values calculated during training (see Figure 1). In order to enable access to such training meta-data, it is thus necessary that an effective XRF accelerator collaborates closely with the host during training, as completely offloading the training task to a specialized accelerator would make explainability impossible. To this end, ACRE offloads and accelerates only AtO comparisons – the very core of the training activity – while leaving the top layers of the training computation to the host. Specifically, we make the following contributions:

- We analyze random forest training, identifying compute and data transfer bottlenecks, and key opportunities to eliminate them.
- We present ACRE, a PIM-based accelerator for HBM systems, co-located with each HBM bank. ACRE significantly reduces the volume of data transferred to the host, eliminating communication bottlenecks. ACRE improves feature value comparison throughput by 538x in HBM3, compared to a baseline system comprising a traditional multicore connected to HBM devices. To the best of our knowledge, ACRE is the first hardware accelerator design that both supports RF training and is specifically tailored for XRF training.
- We evaluate ACRE on multiple datasets and explainable random forest models, finding an average system-level performance improvement for ACRE connected to HBM of 26.6x, compared to a baseline multicore connected to DDR4 memory. ACRE also outperforms an HBM3 baseline solution by 2.5x, increasing to 5x when not bottlenecked by a 16k-thread limit in the host.

This manuscript is organized as follows. In Section 2, we review background material and motivate our work. Section 3 presents the ACRE architecture and its relationship with other HBM components. In Section 4, we evaluate ACRE against baseline solutions. Finally, we summarize related work in Section 5 and conclude.

2 MOTIVATION AND BACKGROUND

2.1 Explainable AI

Explainable AI is an essential axis of future AI deployment for several reasons. First, without XAI features, deployed models risk making the right decisions for the wrong reasons. As an example, [50] describes a situation where some AI models, trained to

detect COVID-19 on chest X-ray images, identified – and made decisions based on – a pattern in the training data: adult X-ray samples had COVID and children samples did not. In practice, these models achieved high accuracy by simply evaluating the size of the skeletal structure; that is, by incorrectly implying that larger skeletons are associated with the presence of COVID. Augmenting the classifications of these models with explanations would have helped pinpoint that the models were classifying patients as COVID-positive or -negative for the wrong reasons.

Second, when AI models are augmented with explanations, they foster users’ trust. Just as patients trust doctors when doctors can clearly explain why they recommend certain procedures, so too, users more willingly accept and use the conclusions of AI systems when they understand the key factors behind such conclusions.

Third, XAI complies with new data legislation that requires enhanced transparency by AI systems. As an example, the European Union’s “General Data Protection Regulation” (GDPR), specifies that end-users have a right to a justification, or explanation, for automated decisions affecting them [16].

Finally, XAI models can offer new task-specific insights. The AI system AlphaGo defeated champion Go player Lee Sedol in 2016. It’s victory in the second game hinged on a mid-game move based on a previously-unexplored game strategy, thus expanding the body of knowledge behind the game.

Given these motivations, advances in AI explainability work to address two distinct goals, local and global explainability. Local explanations show how specific classifications or decisions were reached in the context of a predetermined ML model. In contrast, global explanations offer more general insights into a model’s decision process. For both local and global explainability [34], a popular class of XAI models uses **random forest** algorithms.

2.2 Random Forests

Random Forests (RFs) are a family of machine learning solutions which leverage multiple tree data structures to reach a classification, or recommendation, of data samples. They operate on input organized in tabular form, common in many applications like recommendation systems, with one data sample per row and with features as columns. For tabular input datasets, RFs are highly competitive with deep neural networks, as they maintain high accuracy while offering an intuitive model design [34]. In RF training, many trees are trained independently using distinct random subsets of training data to capture key trends and avoid overfitting. To make an inference, each tree filters the data entry through its nodes, based on the conditions tagging each node, as illustrated in Figure 1. Once the data entry reaches a leaf node (*i.e.*, a node at the bottom of a tree), the output classification associated with that leaf is that tree’s recommendation for that data sample. The overall RF recommendation is calculated as the majority vote among the individual trees’ recommendations. Prior works showed that explanations associated with individual recommendations can be derived from the path traversed in each tree, making random forests a preferred choice for XAI solutions[33] [34] [37] [46] [63].

While RFs have become the primary choice for applications that require explainability, their training is highly memory- and compute-intensive. While neural networks may take 1-2 days to

train on datasets with 15-20M elements, training similarly sized datasets on a random forest of a reasonably-sized set of 1,000 trees takes well over 2 weeks. At such slow rates, training on applications with much larger datasets would take prohibitively long: the training on datasets covering multiple years of US judicial cases or medical visits could easily take 5 or more years [7][51][61]. Moreover, random forests are re-trained many times in production environments, as developers debug, tweak training parameters, and balance the training data for optimal accuracy. Accelerating RF training is therefore an essential step to promulgate XAI adoption among the many application domains that benefit from it. To explore opportunities for RF training acceleration, we must first examine the training process and its bottlenecks.

2.3 RF Training and Bottlenecks

The process of training an RF entails building a set of trees, assigning one feature-based condition (*e.g.*, “Height > 175cm”) to each node. Each tree is trained independently – and thus can be trained concurrently – by using a random subset of the available training dataset. Tree construction starts at the root node and proceeds to the leaves. Each node is trained using whatever subset of the training data reaches it: for instance, if a node’s condition is “Height > 175cm”, one of its children nodes will be trained with all entries where the value of the feature “Height” > 175cm, and the other node with all entries where the same feature has values ≤ 175cm. The condition assigned to the node under construction is selected so that it yields the lowest entropy (least uncertainty among all the possible outcomes, as described below) based on that node’s training data. New children nodes are created for the tree, until all childless nodes are either at the maximum tree-depth, which is user-specified, or no new condition can improve (that is, lower) the entropy. All these childless nodes become the leaves of the tree and are assigned to whichever outcome has a relative majority among the training data that reaches them.

The entropy function H is

$$H = - \sum_{i=1}^n p(x_i) \log_2 p(x_i)$$

where n is the number of possible outcomes in the dataset and $p(x_i)$ is the fraction of data samples that lead to outcome x_i . If most outcomes in a training data subset are identical, the entropy is low. Likewise, if the samples’ outcomes are evenly split across many outcome choices, the entropy is high. After determining how many times each outcome is represented in a data subset, calculating H is computationally simple. However, these values vary for each node, as data samples are further split in constructing each node of the tree, and must be recalculated for every possible condition using all of that node’s training data. The vast majority of operations in computing entropy are **all-to-one (AtO) comparisons**, where *all* feature values are compared against *one* candidate feature condition under consideration. Since each entropy computation executes many AtO comparisons, and entropy is in turn calculated many times for each node (once for each feature value and each feature value in the dataset), training with large datasets easily requires billions of AtO comparisons.¹

¹Some applications use alternate functions instead of entropy, *e.g.*, Gini functions: even these functions require a similar number of AtO comparisons.

RF training bottlenecks. In typical implementations, each tree is trained concurrently, yielding hundreds of parallel threads [17]. In addition, highly-parallel implementations also parallelize entropy calculations for each feature: thus, training a forest of 1,000 trees with a 50-feature dataset yields up to 50,000 parallel active threads, 50 threads for each node being constructed. Prior work [25] demonstrates that parallelizing across all trees and all features in this way offers the highest degree of parallelism available and, by extension, the highest performance. If such massive parallelism could be fully leveraged, it would lead to significant performance improvements; unfortunately, two bottlenecks impact overall performance. First, existing hardware cannot execute many thousands of threads concurrently: modern CPUs are limited to 40-64 cores (commonly equivalent to 80-128 threads) [6][19], and even popular GPUs are limited to a few thousand active threads (e.g., NVIDIA's 3080 Ti at ~10,000 active threads [41]). Even more importantly, memory systems cannot support the vast data transfer bandwidth needed to serve all these threads in parallel.

To better understand these two bottlenecks in RF training, consider a case study like in [43] for medical applications. This study identifies that, in realistic scenarios using RFs, 512 or more trees are trained with datasets comprising 1,000 or more distinct features. Thus, during training, 512,000 threads could theoretically run independently to compute entropy values in parallel. However, supporting so many threads is far beyond the capabilities of any advanced GPU or CPU.

Moreover, training datasets are often too large to fit in cache. Nonetheless, each thread must access large portions of the dataset many times over to compute the entropy function. When serving multiple concurrent threads, the cache thrashes heavily and the repeated accesses to the training data force an extremely high use of the available memory bandwidth, to the point that it becomes the primary bottleneck in the execution. Indeed, the higher the computation parallelism, the higher the memory bandwidth demands. For example, we estimate that if the training algorithm only spawned 512 threads, it would require an estimated 458 GBps of memory bandwidth: assume a 2.4 GHz processor, where each 4-byte operands comparison requires 5 clock cycles, at a 50% cache hit rate ($512 \text{ threads} \times 2.4 \text{B ops/sec} \times 4 \text{ bytes} / 5 \text{ cycles} \times 0.5$). If instead we could fully utilize a much more parallel NVIDIA 3080 Ti GPU (equipped with 10,240 1.37 GHz cores), the bandwidth required would exceed 10 TBps [42]. Figure 2 (in log scale) illustrates our analysis of the number of useful concurrent threads that could carry out comparisons, along with corresponding bandwidth demands for this case study against the resources available in a high-end GPU. Training an RF comprising 256, 512, and 1,024 trees on a dataset with 10k features enables massive parallelism, 2.5M, 5M, and 10M concurrent threads, which in turn entails well over 10 PB/s of data bandwidth. Based on the vast opportunity for massive concurrency, it is clear that solutions for accelerating RF training must address both comparisons' throughput and memory bandwidth bottlenecks in order to deliver meaningful performance improvements.

Motivating study. To validate the bottlenecks of AtO comparisons for RF training, we conducted our own motivating study. We used an Intel Xeon Gold 6126 connected to DDR4-2666 memory to train several RFs while profiling the computation. The findings are shown in the logarithmic plot of Figure 3, indicating that total training

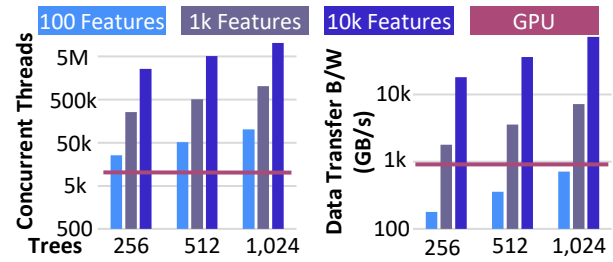


Figure 2: Theoretical concurrent compute resources (left) and memory bandwidth (right) needed to deliver maximum performance of RF training for an industrial-scale medical dataset versus the capabilities of a modern GPU.

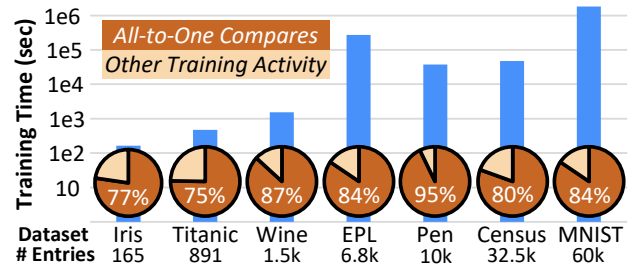


Figure 3: RF training time in Log-scale and relative execution cost of AtO comparisons (100 trees, depth=6).

time is dominated by AtO comparisons involved in the calculation of entropy values. For small datasets (e.g., Iris), all training data fits in cache, leading to minimal data transfers from memory, yet training time is still dominated by AtO comparisons, revealing that the hardware's limited thread concurrency is the primary performance bottleneck. For larger datasets, the heavy data thrashing demands far more bandwidth than memory can service. Note that other works have conducted similar studies and arrived at similar conclusions [8][39][47][62].

Access to AtO results. XRF models use the AtO comparison results to calculate entropy values during forest training, recording how much their confidence in a particular classification increases along each path through the trees. These entropy values are then aggregated to derive explanations in ways that are specific to the explainer function adopted; however, all explainers use the AtO results to compute these relevant values. Therefore, it is essential for XRF accelerators to operate interactively with the host supporting the explainability features, so that these key computation outcomes are not lost or inaccessible to the host.

2.4 High Bandwidth Memory

High Bandwidth Memory (HBM) devices provide wider bandwidth than typical DRAM devices and thus offer a partial solution to the problems described above. HBM devices stack memory layers vertically (3D-stacked) and are tightly integrated with a host die, offering two key benefits: (1) enabling more pins and channels to increase memory bandwidth at similar clock rates, and (2) lowering power consumption by utilizing Through-Silicon Vias (TSVs) to access vertical memory dies, effectively moving data much closer to the host. As a result, HBM consumes 51% lower energy per bit

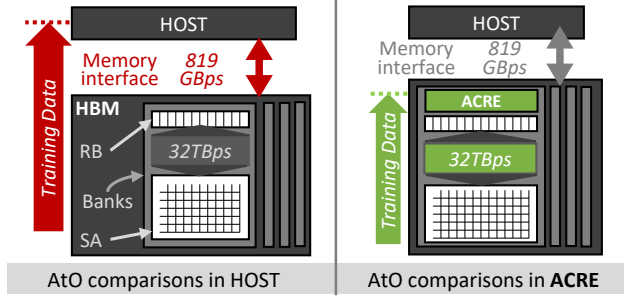


Figure 4: All-to-One (AtO) comparisons on a baseline HBM3 architecture (left) and on the ACRE-augmented HBM3 (right). The baseline must transfer all training data to the host through the memory interface to carry out AtO comparisons. In contrast, ACRE carries out the AtOs in each HBM bank, where internal combined bandwidth between SubArrays (SA) and Row Buffers (RB) is 32 TBps, utilizing the slower external memory interface only for the few-bits of comparison outcomes.

than DDR4 [20][27][26]. While HBMs are a major step forward for memory-bound applications, many of those applications still pose much higher bandwidth demands [12].

In the next section, we highlight opportunities to accelerate RF training without losing access to AtO results and the corresponding entropy values, critical for explainability, and we present methods that leverage these opportunities to improve overall performance.

3 ACRE

ACRE – ACcelerating Random forests for Explainability – is a Processing-in-memory-based (PIM-based) accelerator embedded in HBM memory banks for XRF applications. The key idea of the ACRE accelerator is to include simple hardware within the memory device, specifically within the banks, to execute AtO comparisons efficiently, by taking advantage of both the higher internal (bank-level) bandwidth and the increased comparison throughput that PIM acceleration provides. Figure 4 visualizes the ACRE benefits associated with its co-location with memory. In the baseline solution, when the host executes AtO comparisons (left), all training data is transferred to the host. As training data is often too large to fit in the cache, this data passes through the external memory interface, which, for HBM architectures, operates at a rate of 128–819 GBps, depending on the HBM generation. However, carrying out AtO comparisons within the memory device via ACRE (right) obviates the transfer to the host. The only limit is the hefty internal bandwidth in memory banks: 32TBps with HBM3 devices[22] [27].

The detailed structure of the ACRE architecture is presented in Figure 5. All banks include a series of subarrays (SA) containing memory cells, and a row buffer (RB) which serves as a staging area for data [21]. Training data is spread across the SAs of all banks. To service typical memory read requests from the host, data is first transferred from SAs to RBs with an “Activate” command, then to the host through the memory interface. We propose to augment each HBM bank with an ACRE accelerator unit that contains only three components: a Row Buffer Multiplexer, a set of four comparators, and a Result Accumulator buffer. The host can utilize ACRE

by sending a special memory command (“PIM command”), which includes a reference value. This PIM command asks the ACRE unit to compare all 256 training values (each 32-bits in our XRF implementation) that can fit in the 1KB RB against the reference value and write the results back to memory.

ACRE’s Components. Below, we describe the main ACRE components. The **Row Buffer Multiplexer** (“MUX” in Figure 5) reads directly from the RB and provides the values for AtO comparisons to the comparators in sets of four 32-bit blocks, thus time-multiplexing all 256 32-bit values across 64 cycles (*i.e.*, 2 cycles each [13][36]). **Four comparators** receive these 32-bit values and compare them to the reference value (shown in orange in Figure 5) provided with the PIM command. This reference value is the value used to split all the feature values in calculating an entropy function. Entropy calculations differ between continuous and categorical features: continuous feature data requires determining whether the training data values under analysis are *higher* or *lower* than the reference value, while categorical features require determining whether the training data values *exactly match* the reference value. To accommodate both types of features, each ACRE comparator has two output bits: “greater than” and “equal to.” The eight combined output bits of the four comparators are appended to the **Result Accumulator**, a buffer that holds comparison output bits before they are written back to memory. This buffer is 512 bits wide, so as to accommodate the two output bits of all 256 comparisons, and it is written back to memory once all row buffer data has been analyzed and the accumulator has been filled. These output bits, however, cannot be directly sent from the Result Accumulator to the host: as many ACRE units work in parallel, to do so, each ACRE unit would need additional hardware to synchronize with the other ACRE units and prevent conflicts on the memory bus. Instead, writing back to memory obviates this additional complexity, thus saving precious area. Only eight bits are written to the Result Accumulator simultaneously; the use of shift registers is sufficient to update the writing location of each 8-bit block.

Programming Interface. Only one command is needed to interface with ACRE, keeping both the hardware design and programmability simple. Like all HBM commands, this PIM-compare command supplies both data and an address. The command’s data contains the 32-bit comparison operand against which to compare all values in the row specified in the command’s address. Since ACRE stores the comparison results in memory, the host needs not wait for a response and can send PIM-compare commands to all HBM channels quickly, enabling extremely high throughput as described below.

Maximum Throughput. To understand ACRE’s potential throughput, we explore ACRE’s timing restrictions within HBM devices. Since ACRE takes data from the HBM’s RBs directly, the data bandwidth is limited only by the speed at which RBs are filled. This fill latency is the sum of the row precharge delay ($t_{RP} = 14$ ns), row-to-column delay ($t_{RCD} = 14$ ns) and the cumulative column-to-column delays ($t_{CCDL} = 2.8$ ns). The 1KB RBs are filled in 256b increments from the SAs (*i.e.*, 32 times, once per tCCDL). Finally, to write back, a write-to-precharge delay ($t_{WR} = 14$ ns) is needed before the initial precharge of the next PIM command. We need not include other write-back timing delays as ACRE needs to write only 512 bits: t_{RTP} (read-to-precharge delay), the t_{RP} before writing, and tCCDL delays are masked by precharging the portion of the RB which held

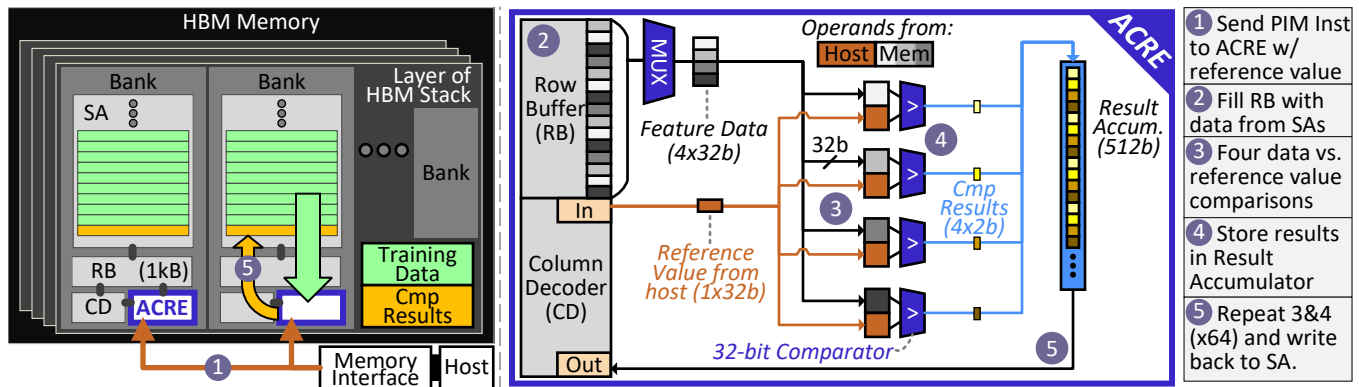


Figure 5: Left: HBM architecture augmented with ACRE, showing a 4-hi stack with banks, Subarrays (SA), Row Buffer (RB), Column Decoder (CD), and ACRE. Right: Internal ACRE architecture to accelerate AtO comparisons. 256 32-bit values, received from the Row Buffer, are compared against a single 32-bit reference value received from the host, time-multiplexed in groups of four. Lastly, all comparators' 2-bit outputs are chained together in the Result Accumulator and written back.

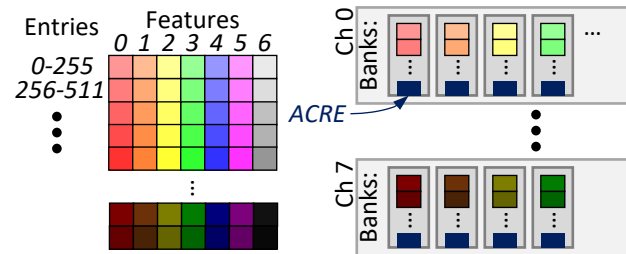


Figure 6: An intuitive organization of the mapping of training values to HBM channels and banks to maximize comparison throughput.

the first few comparison values while ACRE consumes the final RB values during the last of its 256 comparisons. The total fill latency becomes: $14ns + 14ns + (2.8ns * 32) + 14ns = 131.6ns$. At a 1GHz clock, the RB can be refilled every 132 clock cycles. The four comparators together can complete the 256 comparisons in 64 cycles (<132), so performance remains limited only by the RB fill latency. With each of ACRE's 128 units completing 256 comparisons every 131.6 ns, ACRE offers a combined throughput of $\sim 2T$ comparisons per second in HBM3, and even 248B and 500B comparisons per second in HBM1 and HBM2, respectively [58] [59]. Note that the required command bandwidth to achieve such throughput is small: with 16 banks per channel, ACRE only needs to receive one command to each channel every ~ 8 ns ($132/16$), remaining within an HBM architecture's constraints.

To achieve this high compute throughput, PIM-based solutions require close attention to data organization: HBM architectures do not offer communication channels across memory banks, so each ACRE unit has access exclusively to the data in its local bank. As each PIM command compares one operand from the host against all operands in a memory row, each bank must contain training data from one feature only, as shown in Figure 6, to maximize useful comparison throughput. Note that HBM memory controllers already divide the address space such that large contiguous groups of data are split across channels and banks. By storing values of the

same feature in groups of 256 (the number of values which fit in a single memory row), it is trivial to ensure each bank holds only training values of one feature using a simple modulo function. We discuss the positive impact of this organization in Section 4.2.

ACRE does not, however, require this specific data organization. The organization described above is simply the most intuitive option. ACRE can provide its maximum benefit as long as the following conditions are met: (1) training data is partitioned evenly among banks, (2) every memory row with training data contains feature values for a same feature, and (3) the host is aware of which rows store each feature. Any sub-optimal alignment (e.g., if 15 features are divided among 128 banks, such that 7 features have 8 banks and 8 features have 9 banks, or if the final groups of each feature do not fill the 256-value memory rows) will yield only a trivial performance loss, as only a few comparators become underutilized.

Other Applications. While we focus on ACRE's ability to enable XRF applications, the design is applicable to all applications that are bottlenecked by all-to-one comparison throughput. Such applications include sorting, search functions, SQL and other database query applications, finding active vertices in graph and graph database applications, among others.

Additional Benefits of the ACRE Architecture. ACRE's simple design yields three additional benefits. First, as training data never leaves the banks, there are no coherency concerns to address. Second, ACRE places no limits on the trees' size, shape, or organization, nor is it limited to a specific implementation. Performance gains from ACRE will be in addition to any speedups from underlying RF implementations such as SciKit ensemble learning [45], NVIDIA's cuML [49], or algorithmic improvements like those in [10] and [14]. Third, as the operations within ACRE are independent across banks, users are not limited to datasets that fit in a single HBM device: they may have multiple ACRE-equipped HBM devices executing AtO comparisons in parallel, or may swap data from storage devices on-the-fly if needed.

Table 1: Memory Architecture Specifications

Architecture	Bandwidth	Min Latency
DDR4	25.6 GBps	73.3 ns
HBM1 & HBM1+ACRE	128 GBps	106.7 ns
HBM2 & HBM2+ACRE	256 GBps	106.7 ns
HBM3 & HBM3+ACRE	819 GBps	106.7 ns

4 EXPERIMENTAL EVALUATION

4.1 Experimental Setup

XRF Implementation. We developed and use a highly-parallel RF training implementation that takes tabular data with both continuous features (32-bit floating point) and categorical features (one-hot encoded). Each tree is trained concurrently using distinct threads, each spawning its own set of threads, one per feature. These sets of threads collectively execute all comparisons and entropy functions necessary to build each node in the forest, as discussed in Section 2. Each tree trains its nodes sequentially: each thread is assigned all comparisons needed to find the best condition for one feature, such that the total number of concurrent threads is $\#trees \times \#features$.

Architectures. We modeled three memory architectures for our experiments: DDR4, HBM3, and HBM3 equipped with ACRE. Table 1 shows the bandwidth and latency for each architecture. The ACRE memory models are identical to the HBM models, adding one ACRE accelerator unit per bank, as described in Section 3.

Simulators. In order to simulate the ACRE architecture and the two baseline models, we use two simulators, ZSim and Ramulator. ZSim [53] is a cycle-accurate microarchitectural simulator which uses a parallelization technique to scale parallel simulation on many-core hosts efficiently. Ramulator [28] is a cycle-accurate DRAM simulator. We modified prior work [55] which combined ZSim with Ramulator (“ZSim+Ramulator”) to simulate many cores and memory together, extending the implementation to be an event-based simulator modeling PIM units, leading to a highly-accurate (albeit slow) many-core host and memory simulator.

In addition, we developed integrated memory models for ZSim, incorporating both bandwidth and timing constraints of the memory model specifications described above. This enhanced ZSim simulator, which we call ZSim++, enables us to evaluate random forest training on non-trivial datasets with many trees. We validate our ZSim++ solution’s accuracy against ZSim+Ramulator by running a custom memory-bound benchmark, reflecting a workload similar to RF training. Table 2 reports our findings in comparing the accuracy of the two simulators. While the total number of clock cycles differs between ZSim++ and ZSim+Ramulator, the crucial metric to consider in comparing the two simulators is the average number of clock cycles per read access, where we see a <1% difference in simulator behavior. We also observe in our analyses that this difference decreases as workload size increases. As the performance measurements are extremely close between the two simulation infrastructures, and the speed of ZSim++ allows us to build much larger random forests, we use ZSim++ to evaluate ACRE against HBM and DDR baselines in subsequent experiments using ZSim’s OoO core host model (4,096-core, 2.4GHz, 16,384 threads).

Table 2: Accuracy of ZSim++ vs. ZSim+Ramulator

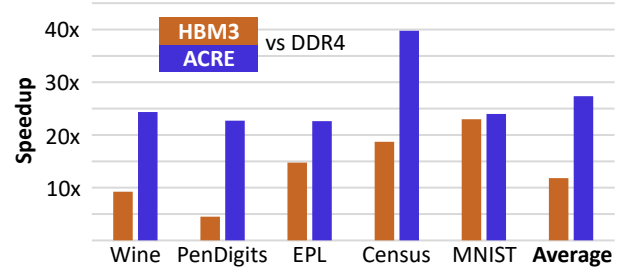
	Clk Cycles	Mem Reads	CC/Rd
ZSim++	600,832	13,022	46.1
ZSim+Ramulator	760,090	16,343	46.5
Variation	26.5%	25.5%	0.87%

Table 3: Training Datasets Properties and Accuracy

Dataset	Features (Cont, Cat)	Entries/Elements	#Trees	Acc.
Wine	11 (11, 0)	1.6k/17k	100	72.6%
Pendigits	16 (16, 0)	11k/165k	100	90.2%
EPL	36 (10, 26)	6.8k/246k	100	80.9%
Census	14 (6, 8)	32k/455k	100	70.5%
MNIST	784 (784, 0)	60k/47M	100	90.3%

Table 4: Relative Speedup of ACRE vs Baselines

Dataset	Memory Tran. (MB)/Node	ACRE on HBM3 Relative Performance	
		DDR	HBM3
Wine	143	24.3x	2.63x
Pendigits	18,839	22.7x	5.04x
EPL	1,717	22.6x	1.53x
Census	4,149	39.7x	2.12x
Mnist	147,927	24x	1.04x

**Figure 7: Relative speedup of ACRE augmenting HBM3, and HBM3 baseline over the DDR baseline.**

Note that core cache size is scaled down to 256kB to maintain a realistic proportion of dataset to cache size.

Evaluation datasets. We evaluate our solution on the five datasets shown in Table 3. These datasets span a broad representation of number of features, their type (with a continuous range, such as between 0-100, or with categorical values, such as true/false), the number of entries (*i.e.*, rows), and total number of data elements in the dataset. We also measure and report the accuracy we obtained on these datasets using a 60/40 split and report the number of nodes in our RF models comprising 100 trees with a maximum depth of 6.

4.2 Performance

With our ZSim++ simulator, we evaluate RF training by simulating a multicore host connected to DDR, one connected to HBM3 equipped with ACRE, and one connected to HBM3 without ACRE. We scale down threads, trees, banks, and external memory bandwidth to limit simulation runtime to a few days and then scale up to the HBM3 bandwidth capacity and to 16,384 threads (consistent with a modern GPU).

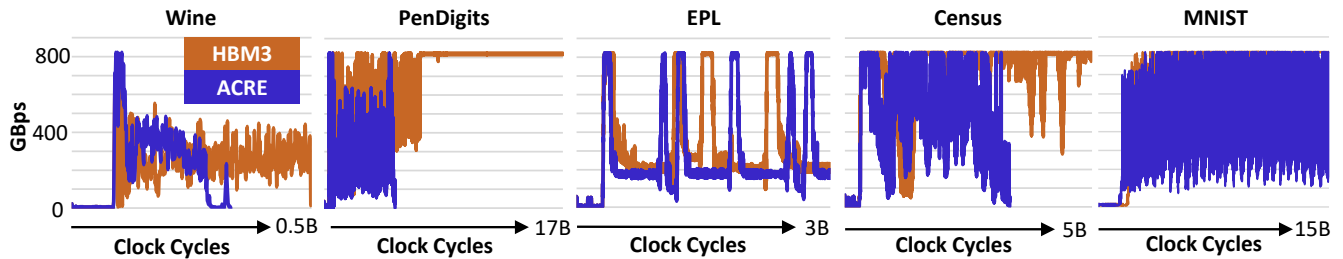


Figure 8: Data bandwidth usage over time when training on the five experimental datasets using the HBM3 baseline and its ACRE-equipped counterpart

Table 4 reports ACRE’s overall performance with respect to the DDR and HBM baselines for our five training datasets: ACRE’s relative speedup against the DDR and HBM3 baselines, as well as the average quantity of data transferred from memory to host. The relative performance against DDR is also plotted in Figure 7. We observe that HBM3 offers an average speedup of 14x vs the DDR baseline. We also observe that ACRE provides an arithmetic mean of 26.6x performance improvement against the DDR baseline and 2.5x against the HBM baseline. ACRE’s performance varies widely between different datasets. Examining the data bandwidth used over time, shown in Figure 8, helps us observe that three factors impact the relative performance improvement: dataset size, tree depth, and the range and distribution of feature values.

The first factor is **dataset size**. ACRE generally accelerates larger datasets more than smaller datasets. When datasets fit in the cache, training is bottlenecked only by parallelism; larger datasets are bottlenecked by both parallelism and memory-host data transfers. Further, larger datasets with more features and entries allow for more sustained parallelism and, by extension, acceleration.

The second factor is the **depth of the trees**. Indeed, not all trees need to reach a maximum depth of six. Shallow trees have more per-node acceleration opportunities than deeper trees, as deep trees become wider and each entry is mapped to only one node per level. With deeper trees, the data is partitioned among more nodes at the same level of a tree, nodes at larger depths have smaller partitions, and the data becomes more sparse. In contrast, portions of the execution operating on nodes closer to the root are associated with denser data partitions, yielding higher row buffer utilization and, thus, higher relative performance.

The third factor is the **range and distribution of feature values**. ACRE performance exceeds that of the baseline solutions when many threads request many comparisons in parallel. Once a thread completes the comparisons for its feature, it must wait to synchronize with other threads training the same node. When all features in the dataset need approximately the same number of comparisons to calculate entropy, threads need not wait long for their node’s coworkers, so thread utilization remains high and ACRE offers higher average throughput. However, when features need vastly different numbers of comparisons, some threads wait for relatively long times to synchronize and ACRE’s throughput is sub-optimal.

The number of comparisons is determined by the number of times a new entropy value is calculated, as we compare every training feature value against a new condition. For categorical data, the number of conditions and entropy calculations is simply the number of categories (e.g., a feature with three categories need

only calculate three entropy values). For continuous data, there are many more possible conditions to test. Binary search across the range of continuous values is used to find a good condition more efficiently. The search stops when entropy does not decrease with more precise condition values. As a result, continuous features with small ranges of values and clear divisions between values in the different classes finish faster than wide ranges with lots of overlap in different classes.

To understand the effect of feature value ranges, consider an example dataset with three categorical features: Features A and B with 2 possible values and Feature C with 10 possible values. For each tree, each feature is assigned one thread that iterates across the dataset entries. Threads for features A and B will finish after only 2 iterations through the feature values, while Feature C’s thread will finish after 10 iterations. This difference in thread workload leads to a 47% thread utilization ($(2+2+10) / 30$) which can severely affect bandwidth and performance when training on some datasets. These three factors affect the bandwidth consumed when training each dataset, each visualized over time in Figure 8. Below we discuss the effects of each of these factors on each dataset.

The Wine dataset [11], the smallest of the five, is comprised entirely of continuous features within a relatively small range of values. Each feature finishes its comparisons at similar times; few threads wait long to synchronize. In addition, trees are shorter on average: there are stronger correlations between feature values and output classes, thus forests stop training at a lower depth. From Figure 8 we can see that wine utilizes a maximum of ~400 GBps after the initial spike. At such bandwidths, HBM3 alone yields 9.2x speedup over DDR4. As wine does not saturate HBM’s bandwidth, the work is compute-bound: adding ACRE to HBM extends the compute capabilities and yields an additional 2.6x over HBM3 alone.

The PenDigits dataset [5] comprises only continuous data, but feature values span a wide range. Training PenDigits without ACRE saturates the available bandwidth, as PenDigits is a much larger dataset and all features have the same value ranges (0-255). As a result, we observe an oscillating bandwidth in Figure 8, rising as many threads launch together, then falling as some threads complete and wait to sync with other threads. With such large bandwidth requirements, HBM3 alone yields 4.5x speedup over DDR4. However, PenDigits training benefits much more from the increased internal data and compute bandwidth that ACRE offers. From Figure 9, which visualizes the amount of memory-to-host data transfer relative to the DDR baseline, we observe that ACRE reduces the data transferred from DDR and HBM memory devices by more than 90%, thus expanding on HBM’s speedup by 5.04x.

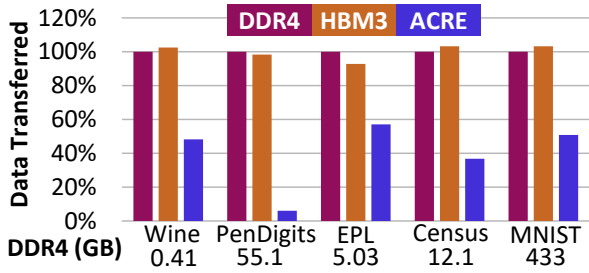


Figure 9: Amount of data transferred from memory (DDR4, HBM, and HBM with ACRE) to host, normalized to the DDR4 baseline.

The EPL dataset [2] presents significant variation between the ranges of feature values. During node training, features with a smaller range of values finish well before those with larger ranges, becoming idle while waiting to sync with the threads of other features, as discussed above. This phenomenon can be observed in the EPL plot of Figure 8: as more threads idle while nodes are training, lower bandwidth and compute throughput is needed, resulting in alternating periods of high and low bandwidth utilization. High average bandwidth enables HBM3 to offer significant performance improvement. ACRE extends this performance gain, but the idle threads limit the upper bound of acceleration. ACRE would offer even higher performance gain if we applied additional parallelization techniques, which have hitherto been unneeded. We leave this study to future work.

Training an RF with the Census dataset [1] experiences similar thread synchronization trends as EPL and PenDigits. However, as training with this larger dataset makes execution more memory-bound, HBM alone offers significant speedups (up to $\sim 18x$). With ACRE's increased compute and bandwidth throughput, training with Census almost saturates HBM's 819 GBps, though we observe some fluctuation due to variations in feature value ranges like with EPL. These benefits combine to achieve $\sim 40x$ speedup over DDR.

Finally, the MNIST dataset [30] is both the largest dataset and has the same range of values across all its 784 features (0-255). These characteristics create very high data bandwidth demand and training quickly becomes memory-bound. HBM3's higher bandwidth results in a 24x speedup over DDR. ACRE offers a modest increase to the HBM3 baseline performance: 25x over DDR. However, as MNIST is a large dataset with many similar features, using ACRE to training on MNIST sees even higher performance gains when not limited by the number of compute threads available in the host, explored further in the scalability analysis below.

As ACRE reduces the amount of data that the host processor needs, we evaluate the total data transferred between the host and the memory system when using ACRE against the DDR and HBM baselines. The results are shown in Figure 9, relative to the DDR baseline. The DDR and HBM baselines have near identical amounts of data transfer to the host, as the host performs the same computations. ACRE, however, saves significant data transfer by sending only 2-bit comparison outputs instead of 32-bit comparison operands. Larger datasets like PenDigits, ones that typically require many memory requests, offer greater opportunity for improvement: $>10x$ reduction in total memory transfer to the host in some cases.

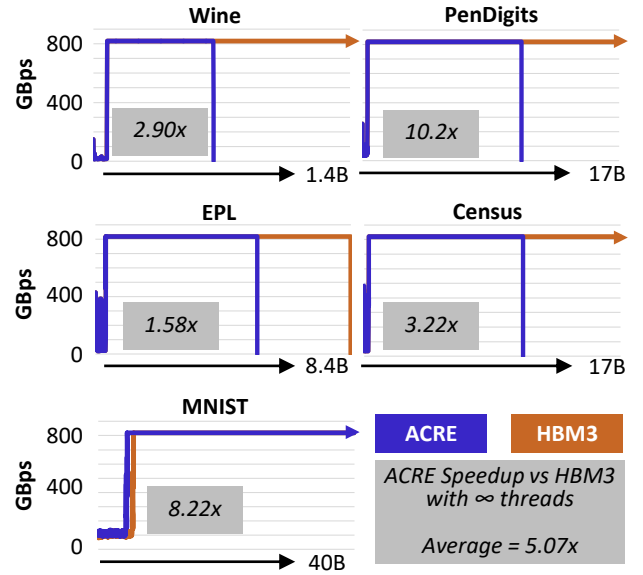


Figure 10: Data bandwidth usage over time and ACRE's relative speedup over HBM3 when paired with host capable of unlimited simultaneous threads

Scalability. Scalability is a critical metric for AI applications, as training dataset sizes continue to grow and as applications are scaled to meet rising demand for automated decision-making. While we use datasets with up to 47M training elements, industry's datasets are often much larger (e.g., medical datasets may have 10,000+ features yielding billions of feature values to compare at each node [43]). Furthermore, forests of at least 512 trees are usually needed to achieve optimal accuracy, and larger forests (i.e., 4,096 trees or more) are sometimes needed [43]. We therefore conduct additional studies to evaluate ACRE's performance when training larger forests with larger datasets and more threads below.

We first note that training with 16,384 threads, consistent with modern GPUs, often does not saturate the available HBM3 external bandwidth of 819 GBps, indicating that additional host compute would yield even higher performance. We therefore evaluate ACRE when training with larger forests, unrestricted by host compute, to understand its performance impact with future, more powerful host hardware, the results of which are shown in Figure 10. We validate that hosts with unlimited cores would saturate the HBM3's data bandwidth when training more trees, shown in the five bandwidth plots. When training on datasets with wide ranges of feature values (Wine, EPL, Census), speedups increase slightly. When datasets have similar feature value ranges, speedups increase even more: ACRE's performance against the HBM3 baseline increases to 8.22x for MNIST and 10.2x for PenDigits.

Further, we compare the maximum possible throughput of AtO comparisons ACRE provides when unbounded by *both* host compute throughput and memory bandwidth for the Census dataset against the throughputs observed when using DDR and HBM3. The results are visualized in Figure 11. Note that ACRE achieves a 538x increase in AtO comparison throughput over the DDR baseline.

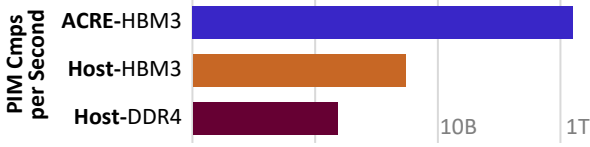


Figure 11: Comparison throughput for DDR4, HBM3 and ACRE when unbounded by host compute throughput for the Census dataset.

We would see similar performance gains from ACRE if RF training algorithms were to include additional software optimizations (e.g., [10], [14]). The benefits shown in our evaluation are a result of the many concurrent comparisons alone, fundamental to any highly-parallel implementation of RF training. In addition, accelerating comparisons could also improve the speed of inference, as inference also demands many similar comparisons. We leave such evaluation as future work.

4.3 Sensitivity and Limitations

While ACRE's performance is dependent on bandwidth, it can still be added to previous generations of HBM. To better understand the impact of using ACRE in memory systems with lower bandwidth, we conducted a sensitivity study to compare HBM1, HBM2, and HBM3 with and without ACRE. Table 5 and Figure 12 show the performance and bandwidth results of this study, respectively. Speedups from ACRE expand by 2-3x for Wine, EPL, and Census from HBM1 to HBM3: ACRE mostly or fully saturated the HBM1 and HBM2 bandwidths, so offering HBM3's 819 GBps throughput allows ACRE to both execute more comparisons in parallel and send more outputs back to the host more quickly. However, ACRE is not always bandwidth-bound when implemented on HBM1 or HBM2. Datasets like Pendigits, as seen in Figure 10, benefit further from more powerful hosts executing more threads in parallel. As a result, increasing the external bandwidth alone does not allow ACRE to increase its performance much further and results in similar relative speedups across HBM1, HBM2, and HBM3 for PenDigits.

As training progresses, nodes at greater depth are trained on increasingly sparse sets of training data: fewer of the 256 comparisons executed on the training data in a row buffer will be needed to calculate entropy, leading to more unused computations. This inefficiency is common across all RF training when data is sufficiently large (i.e., when training data does not fit in the cache) and not unique to ACRE. Future solutions to deep nodes' training data sparsity will complement ACRE and offer higher performance than we observe in our evaluation.

Additionally, ACRE's performance gains are most limited when many threads wait to sync with others, caused by differences in the number of entropy calculations and comparisons to be executed between dataset features (each assigned to a distinct thread). We would expect greater performance improvements if nodes at the same depth in a tree are trained concurrently, reducing synchronization frequency.

4.4 Energy-efficiency Improvement

ACRE reduces system energy by requiring little HBM-host data movement and lowering the host's workload. We model the energy of a single comparison in ACRE as the sum of (a) moving 32 bits

Table 5: Sensitivity Study of ACRE's Relative Performance Improvement on HBM1, HBM2, and HBM3 against DDR4

Dataset	ACRE with:		
	HBM3	HBM2	HBM1
Wine	24.3x	15.7x	9.2x
PenDigits	22.7x	22.2x	18.0x
EPL	22.6x	16.9x	10.1x
Census	39.7x	20.2x	12.8x
MNIST	24x	21.6x	11.8x

Table 6: Energy Consumption for RF training (Census, 55 trees, depth=6)

Memory Model	DDR	HBM3	ACRE
CPU (McPat)	555.3 J	525.7 J	33.9 J
Data Transfer to CPU	13.6 J	1.6 J	-
Data Transfer to ACRE	-	-	0.47 J
ACRE Comparators	-	-	0.0001 J
ACRE Transfer to CPU	-	-	0.098 J
Total Energy	568.9 J	527.2 J	34.5 J
Improvement over DDR	-	7.9%	16.5x

from subarrays (SA) to row buffers (RB), (b) computing a 32-bit comparison, and (c) moving the two output bits from HBM to host. We model the HBM baseline energy as the energy to move 32 bits from SA to host. The calculations assume that many comparisons are occurring in bulk, as they would in RF applications.

A single 64-bit comparator requires 280 fJ per comparison on 180 nm technology [13]. We very conservatively assume the same 280 fJ per 32-bit PIM comparison. Using prior work [56], we scale this energy from 180nm to 20nm technology (0.0335x): 9.3 fJ per comparison. SA-to-RB transfers use 1.2 pJ/bit and SA-to-memory-bus transfers use 3.97 pJ/bit [44]. Note that HBM is approximately an order of magnitude more efficient than DDR (34.42 pJ/bit) [15].

ACRE transfers 32 bits from SAs to RB, executes a 32-bit comparison, and transfers 2 bits from SAs to the host, totaling 46.35 pJ per comparison. In contrast, baseline HBM3 transfers 32 bits from SA to host, which adds up to 127 pJ per comparison. ACRE thus reduces HBM's energy consumption by 2.74x for AtO comparisons, despite executing comparisons in-memory.

We summarize the energy consumption of training a forest (55 trees, depth=6) on the Census dataset for DDR, HBM3, and ACRE in Table 6. McPAT [32] is used for the CPU energy consumption, based on ZSim's reports. HBM3, as a more energy-efficient memory standard, reduces system energy by ~8% against DDR4. However, we observe that ACRE reduces system energy by **15.3x** and **16.5x** vs the HBM3 and the DDR4 baselines, largely due to offloading comparisons to the ACRE units.

4.5 Area

We calculate that <12,000 logic gates are needed in each ACRE unit: thirty-two 150-gate multiplexers, five 200-gate registers (32-bits, four from memory with one reference value), and four 1,500-gate comparators (32-bits) [48]. We extrapolate from [31] that ACRE units, using 20nm technology, would consume 0.043mm² of area. For HBM3, ACRE units together consume ~5% of die area, well

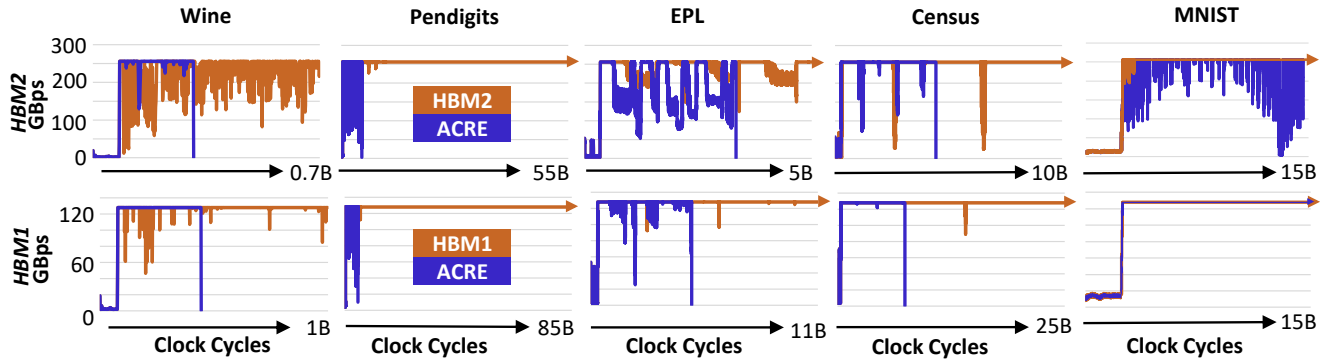


Figure 12: Data bandwidth usage over time across five datasets for HBM2 and HBM1, with and without ACRE

Table 7: Key differences between prior RF-Training accelerators and ACRE

	[57]	[25]	[38]	[24]	ACRE
Accommodates any tree-ensemble training					✓
AtO/Entropy results accessible by host					✓
Accelerates using PIM				✓	✓
Accelerates AtO comparisons	✓	✓	✓	✓	✓
Validates compute & memory bottlenecks	✓	✓	✓	✓	✓

within the area and thermal constraints of HBM standards [31][54]. This silicon footprint is far lower than gate counts observed in prior manufactured solutions by Samsung [31], which use 200,000 gates for logic alone to implement each PIM unit.

5 RELATED WORK

Explainable Random Forests. Developing explainable random forest models remains an active area of research. Some visualize forest patterns, like [40], which converts the paths taken during an RF classification into explicit logic rules which are then visualized in matrix format to clarify model trends to users. iForest [63] expands these visualizations to show the flow and relative weights of decision points. Other papers [52] [46] focus on providing global explanations: descriptions of the forests’ general prediction trends, instead of explanations for specific data points. We focus on local explanations in this work, as local explanations can be used to form global explanations [34].

Random Forest Accelerators. To the best of our knowledge, no XRF hardware accelerator has yet been proposed. Table 7 shows what ACRE contributes beyond what state of the art solutions provide. While no prior work accelerates training without destroying the data needed for explainability, prior RF accelerators have been proposed as follows. [24] proposed regularizing tree shapes, sub-sampling datasets, and inserting analog comparators in DRAM subarrays. They optimize for both prediction accuracy and energy-efficiency, but analog comparators obscure tree paths from software, and thus disqualify many explainers. Our accelerator does not require trees to be the same shape, and works without reducing the number of comparisons. Additionally, some works turn to FPGAs

to address various bottlenecks. [57] designs a random forest classification accelerator, optimizing for small area footprint. Kari, et al. [25] propose a reconfigurable accelerator-based architecture with an efficient data path model for data streaming. [38] also designs an RF accelerator using Altera SDK for OpenCL in order to reduce the hardware size, lower power consumption, and increase the memory bandwidth. [60] compares FPGA-based accelerators with GPU and CPU implementations, concluding that FPGAs offer the highest performance/Watt, but lack scalability. Unfortunately, these prior works would require significant redesign to extract the entropy values that are necessary to build explanations.

PIM Accelerators. Prior works have proposed PIM accelerators within non-HBM 3D-stacked memory technologies. Tesseract [4] proposed adding in-order cores within the logic layer of Hybrid Memory Cubes (HMC), divided among physical bank partitions, to accelerate graph-based processing. DREDGE [35] extended this idea, using hardware to enable live partitioning of dynamic graph data in HMC technology.

Samsung taped-out a general-purpose, programmable PIM accelerator for HBM technology [31], servicing a wider range of applications. While they also chose a bank-level accelerator similar to ACRE, they sacrificed half the device’s memory capacity to accommodate a much larger design and can only fit one PIM unit per two banks. Aga [3] proposed a near-data processing unit for ML workloads on HBM devices. Other works have aimed to reduce the area overhead of general-purpose PIM accelerators. Newton [18] proposes a PIM accelerator for machine learning workloads that accelerates matrix-vector multiplication with MAC units placed near DRAM banks. Kwon [29] designed a near-memory processing architecture to increase memory capacity and bandwidth for embedding gathers and tensor operations. While these designs use bank-level ALUs that work well for general matrix multiplication (GEMM), they consume much more precious area and their multiple commands make them more complex to program than ACRE.

6 CONCLUSION

As more applications employ AI-based solutions, explainability is quickly becoming a critical component of AI models in industry. Explainable Random forest (XRF) models have emerged as a key solution, but training these models is prohibitively slow for all but very small workloads: faster methods to train explainable random forests are needed.

In response to these problems, we present ACRE, a PIM-based accelerator placed at each bank of HBM devices to execute the All-to-one (AtO) comparisons that dominate RF training time. To the best of our knowledge, we are the first to present a hardware accelerator specific to XRF training. Using only a small fraction of area that can be used for bank-level logic, ACRE yields an overall system-level training performance improvement of 26.6x on average, compared to a baseline multicore processor solution with DDR4 memory. Further, ACRE yields a 2.5x improvement when compared to an HBM3 architecture baseline, increasing to 5x when not bottlenecked by a 16k-thread limit in the host. In addition, due to the much higher performance, we observe that our solution also provides a 16.5x energy reduction overall, over a DDR baseline. For future work, we hope to explore ACRE's performance when using additional parallelization techniques which have hitherto been unneeded, as well as opportunities to accelerate RF inference in addition to training.

ACKNOWLEDGMENTS

We would like to thank the anonymous reviewers for their thorough feedback and generous guidance.

This work was supported by the Applications Driving Architectures (ADA) Research Center, a JUMP Center co-sponsored by SRC and DARPA.

REFERENCES

- [1] [n. d.]. Adult Census Dataset. <https://archive.ics.uci.edu/ml/datasets/adult>.
- [2] [n. d.]. English Premier League (EPL) Results. www.kaggle.com/datasets/irkaal/english-premier-league-results.
- [3] S. Aga, N. Jayasena, and M. Ignatowski. 2019. Co-ML: a case for Co llaborative ML acceleration using near-data processing. In *Proc. MEMSYS*.
- [4] J. Ahn, S. Hong, S. Yoo, O. Mutlu, and K. Choi. 2015. A scalable processing-in-memory accelerator for parallel graph processing. In *Proc. ISCA*.
- [5] F. Alimoglu and E. Alpaydin. 1996. Methods of combining multiple classifiers based on different representations for pen-based handwritten digit recognition. In *Proc. TAINN*.
- [6] AMD. 2020. 3990X Specifications. www.amd.com/en/product/9111.
- [7] American Hospital Association. 2021. Total number of hospital outpatient visits in the U.S. from 1965 to 2019. <https://www.statista.com/statistics/459744/total-outpatient-visit-numbers-in-the-us/>.
- [8] C. Cheng and C. Bouganis. 2013. Accelerating random forest training process using FPGA. In *Proc. FPL*. IEEE.
- [9] M. Chirodea, O. Novac, C. Novac, N. Bizon, M. Oproescu, and C. Gordan. 2021. Comparison of tensorflow and pytorch in convolutional neural network-based applications. In *Proc. ECAI*.
- [10] S. Choi, D. Ko, S. Hwang, and Y. Choi. 2018. Memory-efficient random forest generation method for network intrusion detection. In *Proc. ICUFN*.
- [11] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis. 2009. Modeling wine preferences by data mining from physicochemical properties. *Decision Support Systems* (2009).
- [12] A. Dakkak, C. Li, J. Xiong, I. Gelado, and W. Hwu. 2019. Accelerating Reduction and Scan Using Tensor Core Units. In *Proc. ISC*.
- [13] M. Ghasemzadeh, S. Najafbisfar, and A. Amini. 2018. Ultra Low-power, High-speed Digital Comparator. In *Proc. MIXDES*.
- [14] F. Gieseke and C. Igel. 2018. Training big random forests with little resources. In *Proc. SIGKDD*.
- [15] B. Giridhar, M. Cieslak, D. Duggal, R. Dreslinski, H. Chen, R. Patti, B. Hold, C. Chakrabarti, T. Mudge, and D. Blaauw. 2013. Exploring DRAM Organizations for Energy-Efficient and Resilient Exascale Memories. In *Proc. SC*.
- [16] B. Goodman and S. Flaxman. 2017. European Union Regulations on Algorithmic Decision-Making and a "Right to Explanation". *AI Magazine* (2017).
- [17] H. Grahm, N. Lavesson, M. H. Lapajne, and D. Slat. 2011. CudaRF: A CUDA-based implementation of Random Forests. In *Proc. AICCSA*.
- [18] M. He, C. Song, I. Kim, C. Jeong, S. Kim, I. Park, M. Thottethodi, and T. Vijaykumar. 2020. Newton: A DRAM-maker's Accelerator-in-Memory (AiM) Architecture for Machine Learning. In *Proc. MICRO*.
- [19] Intel. 2021. Xeon 8380 Specifications. www.intel.com/content/www/us/en/products/sku/212287/intel-xeon-platinum-8380-processor-60m-cache-2-30-ghz/specifications.html.
- [20] JEDEC. 2021. DDR4 SDRAM STANDARD. (2021).
- [21] JEDEC. 2021. High Bandwidth Memory (HBM) DRAM. (2021).
- [22] JEDEC. 2023. High Bandwidth Memory (HBM3) DRAM. (2023).
- [23] Kaggle. 2021. State of Data Science and Machine Learning. <https://www.kaggle.com/kaggle-survey-2021>
- [24] M. Kang, S. Gonugondla, S. Lim, and N. Shanbhag. 2018. A 19.4-nJ/decision, 364-K decisions/s, in-memory random forest multi-class inference accelerator. *IEEE Trans. SSC* (2018).
- [25] T. Kari, N. Leelavani, R. Dhanushree, K. Jagannatha, and S. Natarajan. 2021. An Accelerated Approach to Parallel Ensemble Techniques Targeting Healthcare and Environmental Applications. In *Proc. ICEPE*.
- [26] J. Kim and Y. Kim. 2014. HBM: Memory solution for bandwidth-hungry processors. In *Proc. HCS*.
- [27] J. H. Kim, S. Kang, S. Lee, H. Kim, W. Song, Y. Ro, S. Lee, D. Wang, H. Shin, B. Phuath, J. Choi, J. So, Y. Cho, J. Song, J. Choi, J. Cho, K. Sohn, Y. Sohn, K. Park, and N. Kim. 2021. Aquabolt-XL: Samsung HBM2-PIM with in-memory processing for ML accelerators and beyond. In *Proc. HCS*.
- [28] Y. Kim, W. Yang, and O. Mutlu. 2016. Ramulator: A Fast and Extensible DRAM Simulator. *IEEE Computer Architecture Letters* (2016).
- [29] Y. Kwon, Y. Lee, and M. Rhu. 2019. Tensordimm: A practical near-memory processing architecture for embeddings and tensor operations in deep learning. In *Proc. MICRO*.
- [30] Y. LeCun, C. Cortes, and C. Burges. 2010. MNIST handwritten digit database. (2010).
- [31] S. Lee, S. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, O. Seongil, A. Iyer, D. Wang, K. Sohn, and N. S. Kim. 2021. Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology: Industrial Product. In *Proc. ISCA*.
- [32] S. Li, J. Ahn, R. Strong, J. Brockman, D. Tullsen, and N. Jouppi. 2009. McPAT: An integrated power, area, and timing modeling framework for multicore and manycore architectures. In *Proc. MICRO*.
- [33] K. P. Lu Thi, N. C. Vo Thi, and N. H. Phung. 2015. Extracting Rule RF in Educational Data Classification: From a Random Forest to Interpretable Refined Rules. In *Proc. ACOMP*.
- [34] S. Lundberg, G. Erion, H. Chen, A. DeGrave, J. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S. Lee. 2020. From local explanations to global understanding with explainable AI for trees. *Nature machine intelligence* (2020).
- [35] A. McCrabb, E. Winsor, and V. Bertacco. 2019. Dredge: Dynamic repartitioning during dynamic graph execution. In *Proc. DAC*.
- [36] E. Menendez, D. Maduik, R. Garg, and S. Khatri. 2006. CMOS comparators for high-speed and low-power applications. In *Proc. ICCD*.
- [37] I. Mollas, N. Bassiliades, I. Vlahavas, and G. Tsoumakas. 2019. LionForests: local interpretation of random forests. *arXiv* (2019).
- [38] H. Nakahara, A. Jingui, T. Fujii, and S. Sato. 2016. An acceleration of a random forest classification using Altera SDK for OpenCL. In *Proc. FPT*.
- [39] R. Narayanan, D. Honbo, G. Memik, A. Choudhary, and J. Zambreno. 2007. An FPGA implementation of decision tree classification. In *Proc. DATE*.
- [40] M. Neto and F. Paulovich. 2020. Explainable matrix-visualization for global and local interpretability of random forest classification ensembles. *IEEE Trans. Vis. Comput. Graph.* (2020).
- [41] NVIDIA. 2017. 1080Ti User Guide. https://www.nvidia.com/content/geforce-gtx/GTX_1080_User_Guide.pdf.
- [42] NVIDIA. 2021. 3080Ti. <https://www.nvidia.com/en-us/geforce/graphics-cards/30-series/rtx-3080-3080ti/>.
- [43] T. Oshiro, P. Perez, and J. Baranauskas. 2012. How many trees in a random forest?. In *Proc. MLDM*.
- [44] M. O'Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. Keckler, and W. Dally. 2017. Fine-grained DRAM: Energy-efficient DRAM for extreme bandwidth systems. In *Proc. MICRO*.
- [45] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *JMLR* (2011).
- [46] D. Petkovic, R. Altman, M. Wong, and A. Vigil. 2018. Improving the explainability of Random Forest classifier—user centered approach. In *Proc. PSB*.
- [47] P. Poorheravi and V. Gaudet. 2022. FPGA-Based Architectures for Random Forest Acceleration. In *Proc. MWSCAS*.
- [48] K. Prabhala and P. Raju. 2019. An Implementation of 32 BIT CMOS Comparator in Mentor EDA Tools. *IJEE* (2019).
- [49] S. Raschka, J. Patterson, and C. Nolet. 2020. Machine learning in python: Main developments and technology trends in data science, machine learning, and artificial intelligence. *Information* (2020).
- [50] M. Roberts, D. Driggs, M. Thorpe, J. Gilbey, M. Yeung, S. Ursprung, A. Aviles-Rivero, C. Etmann, C. McCague, and L. Beer. 2021. Common pitfalls and recommendations for using machine learning to detect and prognosticate for COVID-19 using chest radiographs and CT scans. *Nature Machine Intelligence* (2021).

- [51] SafeAtLast. 2023. Lawsuit Statistics To Intrigue You in 2023. <https://safeatlast.co/blog/lawsuit-statistics>.
- [52] O. Sagi and L. Rokach. 2020. Explainable decision forest: Transforming a decision forest into an interpretable tree. *Trans. IF* (2020).
- [53] D. Sanchez and C. Kozyrakis. 2013. ZSim: Fast and accurate microarchitectural simulation of thousand-core systems. *SIGARCH Computer Arch. News* (2013).
- [54] A. Shilov. 2016. JEDEC Publishes HBM2 Specification as Samsung Begins Mass Production of Chips. <https://www.anandtech.com/show/9969/jedec-publishes-hbm2-specification>.
- [55] G. Singh, J. Gómez-Luna, G. Mariani, G. Oliveira, S. Corda, S. Stuijk, O. Mutlu, and H. Corporaal. 2019. Napel: Near-memory computing application performance prediction via ensemble learning. In *Proc. DAC*.
- [56] A. Stillmaker and B. Baas. 2017. Scaling equations for the accurate prediction of CMOS device performance from 180nm to 7nm. *Trans. Integration* (2017).
- [57] R Struharik. 2015. Decision tree ensemble hardware accelerators for embedded applications. In *Proc. Sisy*.
- [58] Synopsys. 2022. HBM3 Controller. https://www.synopsys.com/dw/doc.php/ds/c/dwc_hbm3_controller_ds.pdf.
- [59] K. Tran. 2016. The era of high bandwidth memory. In *Proc. HCS*.
- [60] B. Van Essen, C. Macaraeg, M. Gokhale, and R. Prenger. 2012. Accelerating a Random Forest Classifier: Multi-Core, GP-GPU, or FPGA?. In *Proc. FCCM*.
- [61] M. Wright and A. Ziegler. 2017. Ranger: A Fast Implementation of Random Forests for High Dimensional Data in C++ and R. *Trans. JSS* (2017).
- [62] L. Zhao, Q. Deng, Y. Zhang, and J. Yang. 2019. RFacc: A 3D ReRAM associative array based random forest accelerator. In *Proc. ISC*.
- [63] X. Zhao, Y. Wu, D. Lee, and W. Cui. 2019. iForest: Interpreting Random Forests via Visual Analytics. *IEEE Trans. Vis. Comput. Graph.* (2019).